Bubble sort

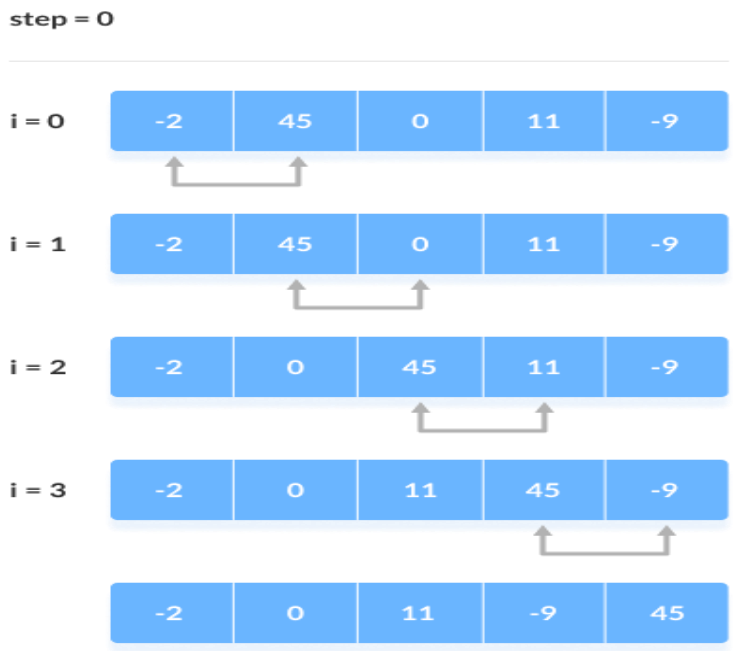
Bubble sort is a sorting algorithm that compares two adjacent elements and swaps them until they are in the intended order.

Just like the movement of air bubbles in the water that rise up to the surface, each element of the array move to the end in each iteration. Therefore, it is called a bubble sort.

Working of Bubble Sort: Suppose we are trying to sort the elements in **ascending order**.

1. First Iteration (Compare and Swap)

1. Starting from the first index, compare the first and the second elements.
2. If the first element is greater than the second element, they are swapped.
3. Now, compare the second and the third elements. Swap them if they are not in order.
4. The above process goes on until the last element.

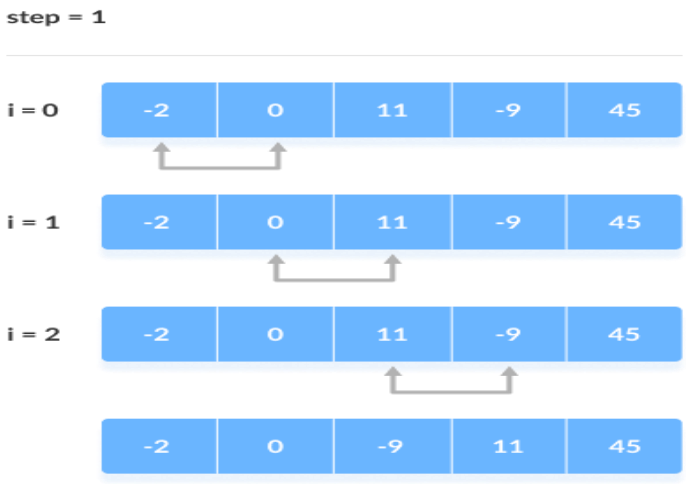


Compare the Adjacent Elements

2. Remaining Iteration

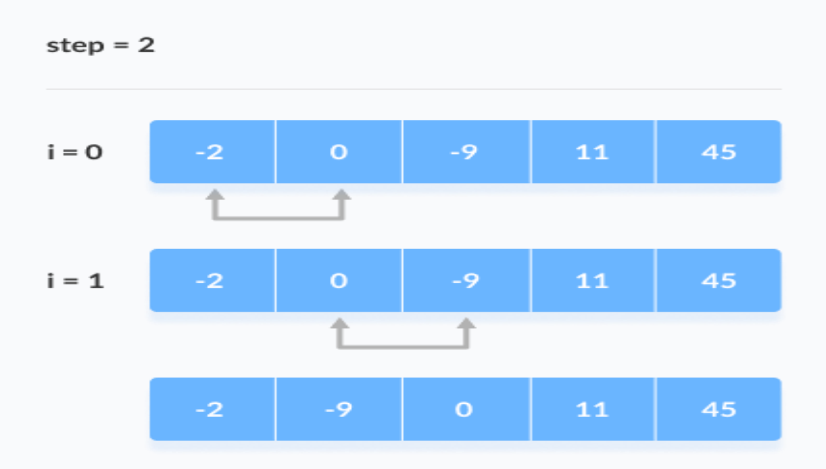
The same process goes on for the remaining iterations.

After each iteration, the largest element among the unsorted elements is placed at the end.



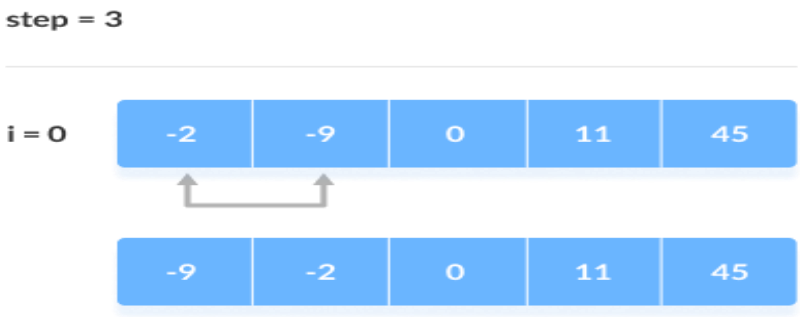
Put the largest element at the end

In each iteration, the comparison takes place up to the last unsorted element.



Compare the adjacent elements

The array is sorted when all the unsorted elements are placed at their correct positions.



The array is sorted if all elements are kept in the right order

Bubble Sort Complexity

Time Complexity	
Best	O(n)
Worst	$O(n^2)$
Average	$O(n^2)$
Space Complexity	O(1)

Complexity in Detail

Bubble Sort compares the adjacent elements.

Cycle	Number of Comparisons
1st	(n-1)
2nd	(n-2)
3rd	(n-3)
.....	.....
last	1

Hence, the number of comparisons is **(n-1) + (n-2) + (n-3) +.....+ 1 = n(n-1)/2**
nearly equals to n^2
Hence, **Complexity:** $O(n^2)$
Also, if we observe the code, bubble sort requires two loops. Hence, the complexity is $n*n = n^2$

1. Time Complexities

Best Case Complexity:O(n): If the array is already sorted, then there is no need for sorting.

Worst Case Complexity:O(n^2): If we want to sort in ascending order and the array is in descending order then the worst case occurs.

Average Case Complexity:O(n^2): It occurs when the elements of the array are in jumbled order (neither ascending nor descending).

2. Space Complexity

Space complexity isO(1) because an extra variable is used for swapping.
In the **optimized bubble sort algorithm**, two extra variables are used. Hence, the space complexity will be O(2)

Bubble Sort Applications

Bubble sort is used if

- complexity does not matter
- short and simple code is preferred

****

Bubble Sort Implementation

To implement the Bubble Sort algorithm in a programming language, we need:

1. An array with values to sort.
2. An inner loop that goes through the array and swaps values if the first value is higher than the next value. This loop must loop through one less value each time it runs.
3. An outer loop that controls how many times the inner loop must run. For an array with n values, this outer loop must run n-1 times.

```
a = [7, 3, 9, 12, 11]

n = len(a)
for i in range(0, n-1):
    print("Pass",i)
    for j in range(0, n-i-1):
        if a[j] > a[j+1]:
            temp=a[j]
            a[j]=a[j+1]
            a[j+1]=temp
    print("Sorted array:", a)
```

Output:

```
Pass 0
Sorted array: [3, 7, 9, 11, 12]
Pass 1
Sorted array: [3, 7, 9, 11, 12]
Pass 2
Sorted array: [3, 7, 9, 11, 12]
Pass 3
Sorted array: [3, 7, 9, 11, 12]
```

****

```
a = [7, 3, 9, 12, 11]


n = len(a)
for i in range(0, n-1):
    print("Pass",i)
    falg = 0
    for j in range(0, n-i-1):
        if a[j] > a[j+1]:
            temp=a[j]
            a[j]=a[j+1]
            a[j+1]=temp
            flag=1
        print("Sorted array:", a)
    if flag==0:
            break
```

Output:

```
Pass 0
Sorted array: [3, 7, 9, 12, 11]
Sorted array: [3, 7, 9, 12, 11]
Sorted array: [3, 7, 9, 12, 11]
Sorted array: [3, 7, 9, 11, 12]
Pass 1
Sorted array: [3, 7, 9, 11, 12]
Sorted array: [3, 7, 9, 11, 12]
Sorted array: [3, 7, 9, 11, 12]
Pass 2
Sorted array: [3, 7, 9, 11, 12]
Sorted array: [3, 7, 9, 11, 12]
Pass 3
Sorted array: [3, 7, 9, 11, 12]
```

****

Bubble Sort Improvement

In the above algorithm, all the comparisons are made even if the array is already sorted.

This increases the execution time.

To solve this, we can introduce an extra variable swapped. The value of swapped is set true if there occurs swapping of elements. Otherwise, it is set **false**.
After an iteration, if there is no swapping, the value of swapped will be **false**. This means elements are already sorted and there is no need to perform further iterations.
This will reduce the execution time and helps to optimize the bubble sort.

```
a = [7, 3, 9, 12, 11]

n = len(a)
for i in range(n-1):
    print("Pass:",i)
    swapped = False
    for j in range(n-i-1):
        if a[j] > a[j+1]:
            temp=a[j]
            a[j]=a[j+1]
            a[j+1]=temp
            swapped = True
        print("Sorted Array:",a)
    if not swapped:
        break

print("Sorted array:", a)
```

Output:

```
Pass: 0
Sorted Array: [3, 7, 9, 12, 11]
Sorted Array: [3, 7, 9, 12, 11]
Sorted Array: [3, 7, 9, 12, 11]
Sorted Array: [3, 7, 9, 11, 12]
Pass: 1
Sorted Array: [3, 7, 9, 11, 12]
Sorted Array: [3, 7, 9, 11, 12]
Sorted Array: [3, 7, 9, 11, 12]
Sorted array: [3, 7, 9, 11, 12]
```